
🔥 CLAUDE PROFESSIONAL ARCHITECT — MASTER NOTES

🧠 0. HOW TO THINK IN THIS EXAM (MOST IMPORTANT)

Every question is testing this:

👉 **"Do you understand deterministic vs probabilistic control?"**

Approach	Reliability	When to use
Prompt-based	❌ Probabilistic	Guidance only
Few-shot	⚠️ Better but still probabilistic	Pattern shaping
Tool use	✅ Structured	Output control
Hooks / Programmatic rules	✅ Deterministic	Critical logic

👉 Golden Rule:

If failure has business impact → NEVER rely on prompts

🧩 1. AGENTIC ARCHITECTURE (27%) — CORE OF EXAM

🔁 Agent Loop (CRITICAL)

Lifecycle:

1. Send request to Claude
2. Check `stop_reason`
   - `"tool_use"` → call tool → append result → continue
   - `"end_turn"` → STOP

👉 MUST KNOW:

❌ Anti-patterns:

- Checking text like “done”
- Limiting iterations arbitrarily
- Parsing natural language to decide flow

✔ Correct:

- **Always rely on `stop_reason`**

🧠 Model-driven vs Hard-coded

Type	Behavior
Model-driven	Claude decides next action
Deterministic	You enforce sequence

👉 Exam trick:

- If order matters → **programmatic enforcement**

🧠 Multi-Agent Design (VERY IMPORTANT)

Architecture:

👉 Coordinator (brain) + Subagents (workers)

- Subagents DO NOT share memory
- Coordinator handles:
  - decomposition
  - routing
  - aggregation
  - retries

🚨 Most common exam trap:

Coordinator decomposes task TOO NARROW

Example from exam:

- Only “visual arts” → misses music, film

👉 Root cause:
Bad decomposition, not bad subagents

⚙️ Subagent Invocation

- Uses **Task tool**
- Must include `allowedTools: ["Task"]`

👉 Key rules:

✔️ Pass context explicitly
✔️ Include ALL relevant findings
✔️ Use structured format

❌ Never assume memory sharing

🔀 Parallel Execution

👉 Best optimization:

- Emit multiple `Task` calls in ONE response

NOT:

- sequential delegation

🔐 Enforcement vs Prompt

Requirement	Solution
Must happen ALWAYS	Hooks / blocking
Usually should happen	Prompt

👉 Example:

❌ Prompt: “Always verify user”
✔️ Hook: block refund unless verified

🧩 Hooks (VERY TESTED)

Used for:

1. **Intercept tool results**
2. **Block tool calls**

Example:

- Refund > $500 → block → escalate

🧠 Task Decomposition Strategy

Problem Type	Approach
Predictable	Prompt chaining
Unknown / research	Dynamic decomposition

🔄 Sessions

Feature	Use
`--resume`	Continue session
`fork_session`	Try alternatives

👉 Trick:

- If data stale → **start new session with summary**

🔧 2. TOOL DESIGN & MCP (18%)

🎯 Tool Description = MOST IMPORTANT

👉 This is how model selects tools

Bad:

“Get order”

Good:

- Inputs
- Examples
- Boundaries
- When to use vs others

🚨 Classic Exam Question

Problem:

- Wrong tool selected

Solution:
👉 Improve **tool descriptions FIRST**

NOT:

- Few-shot
- Classifier
- New architecture

⚠️ Tool Overload

👉 Golden Rule:

More tools = worse accuracy

✔️ Ideal:

- 4–5 tools per agent

🧱 Error Handling (VERY IMPORTANT)

Must return:

```
{
  "isError": true,
  "errorCategory": "transient | validation | permission | business",
  "isRetryable": true/false
}
```

🚨 Anti-patterns:

❌ "Operation failed"
❌ Returning empty results
❌ Killing entire workflow

🧠 Tool Distribution

Agent	Tools
Search	search tools
Synthesis	minimal tools

👉 NEVER give all tools to all agents

🎛️ tool_choice

Option	Behavior
auto	model may not call tool
any	MUST call a tool
forced	MUST call specific tool

⚙️ 3. CLAUDE CODE (20%)

🧾 CLAUDE.md Hierarchy

Level	Scope
User (~/.claude)	personal
Project	shared
Directory	local

👉 Exam trap:

- Team not seeing config → stored at user level

📂 Rules System

Best approach:

```
paths:
  - "**/*.test.tsx"
```

👉 Why?

- Applies across folders

⚡ Slash Commands

Location	Scope
`.claude/commands/`	team
`~/.claude/commands/`	personal

🧠 Skills

Used for:

- reusable workflows

Important configs:

- `context: fork`
- `allowed-tools`

🧠 Plan Mode vs Direct

Use Plan Mode when	Use Direct when
architecture	small change
many files	single file
unknown approach	known fix

👉 ALWAYS choose plan mode for:

- migrations
- restructuring

🔄 Iteration Strategy

Best method:

1. Write tests
2. Run
3. Fix failures

🚀 CI/CD

Use:

```
claude -p "prompt"
```

👉 Without `-p` → it hangs

📦 Structured Output

Use:

```
--output-format json
--json-schema
```

🧠 4. PROMPT ENGINEERING (20%)

🎯 Precision > Vague Instructions

Bad:

“Check accuracy”

Good:

“Flag when comments contradict code behavior”

🔥 Few-Shot (VERY IMPORTANT)

Use for:

- ambiguous cases
- formatting consistency

👉 Best: 2–4 examples

📦 Structured Output

👉 BEST method:

tool_use + JSON schema

⚠️ Important:

- Fixes syntax errors
- NOT semantic errors

🔄 Retry Loop

When fails:

1. Send original input
2. Include error
3. Retry

🧠 When retry fails:

👉 When data DOESN’T exist

📊 Batch Processing

Use batch when	Don't use when
async jobs	real-time
reports	pre-merge checks

🧠 Multi-Pass Review (VERY TESTED)

Instead of:

❌ analyze all files together

Do:

✔ per-file pass
✔ integration pass

🧠 5. CONTEXT MANAGEMENT (15%)

🧠 Biggest Risk

“Lost in the middle”

👉 Model forgets middle content

🧾 Best Practice

Create:

👉 **Case Facts Block**

- IDs
- amounts
- dates

🧹 Context Optimization

✔ trim tool outputs
✔ keep relevant fields only
✔ structure data

🚨 Escalation Rules

Escalate when:

- user asks
- policy unclear
- no progress

❌ Wrong signals:

- sentiment
- confidence score

⚠️ Error Propagation

Best:

```
{
  "failure_type": "...",
  "attempted_query": "...",
  "partial_results": [...]
}
```

🧠 Codebase Exploration

Best flow:

1. Grep → find entry
2. Read → understand
3. Expand gradually

👤 Human Review

Use:

- confidence scores
- stratified sampling

📌 Provenance (VERY IMPORTANT)

Always include:

- source
- date
- evidence

⚠️ Conflict Handling

Never:

- pick one source

Always:

- show both

🎯 FINAL EXAM STRATEGY

🧠 How to eliminate wrong answers

❌ Remove answers that:

- rely only on prompt
- introduce unnecessary infra
- hide errors
- overcomplicate

✅ Choose answers that:

- enforce determinism

- improve structure
- reduce ambiguity
- match real-world constraints

🔥 MOST COMMON EXAM TRAPS

---

1. Prompt vs Programmatic → choose programmatic
2. Tool issue → fix description first
3. Multi-agent issue → coordinator fault
4. Context issue → structure data
5. Large task → multi-pass
6. CI/CD → use `-p`
7. Batch API misuse → latency mismatch

🚀 WHAT YOU SHOULD DO NEXT

🔥 Phase 2 (HIGH IMPACT)

- 3 FULL mock exams (real weightage)
- scenario-based traps
- answer explanations

🔥 Phase 3

- Hands-on mini projects (fastest way to lock concepts)

🔥 Phase 4

- Rapid revision sheet (1-page memory dump before exam)
